StudySync — Peer Matching & Collaborative Study Group Platform

System Design: Architecture Diagram & Technical Specification

Course: Software Engineering & Project Management

Project Group: Group 15 (Section F)

Target System: StudySync Web Platform

Date of Release: October 16, 2026

Architectural Style: Three-Tier Layered Client-Server Architecture

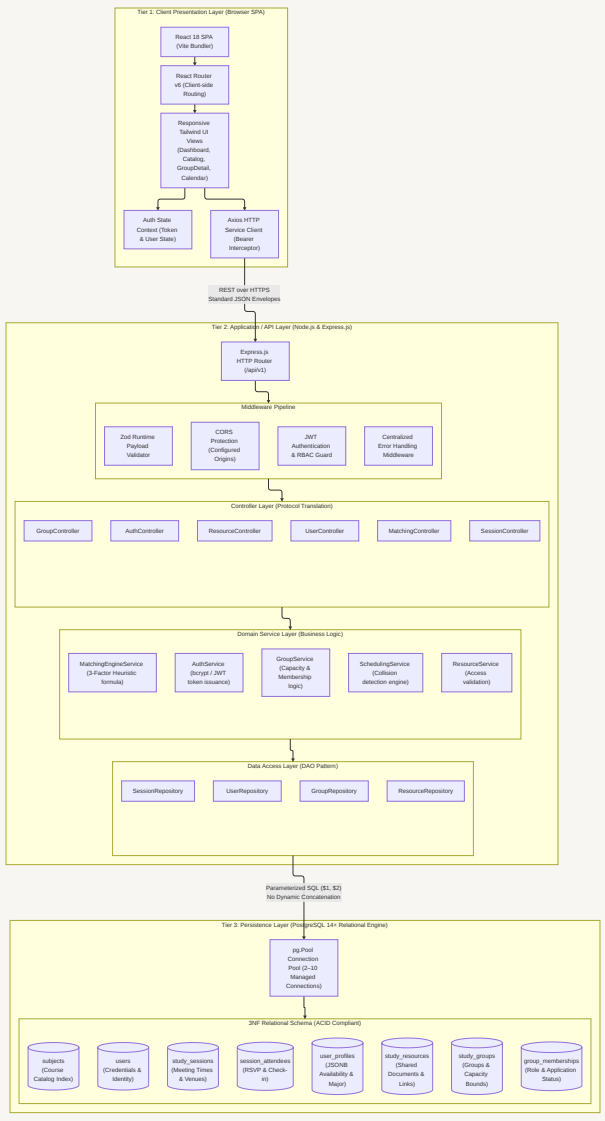
1. Architectural Overview & Design Principles

StudySync is architected as a **Three-Tier Layered Architecture** adhering to clean separation of concerns, unidirectional control flow, and interface-driven decoupling.

Core Architectural Principles:

1. **Separation of Concerns:** The presentation layer has no knowledge of SQL queries; controllers handle only HTTP protocol mapping; services execute domain logic; repositories manage persistence.
2. **Stateless Scalability:** The application tier preserves zero session state in server memory. User authentication relies on cryptographically signed JSON Web Tokens (JWT), allowing horizontal scaling without session replication.
3. **Defense in Depth:** Input validation occurs at the client boundary (form constraints), the HTTP gateway (declarative Zod schemas), and the database engine (3NF check constraints and foreign keys).
4. **Relational Rigor:** Data integrity is guaranteed via PostgreSQL ACID transactions, foreign key cascading rules, and composite unique keys.

2. Master System Architecture Diagram



3. Layer-by-Layer Decomposition & Responsibilities

3.1 Tier 1: Presentation Layer (Client SPA)

- **Framework & Language:** React 18 with TypeScript (`.tsx`), built and bundled using Vite.
- **Styling Engine:** Tailwind CSS providing utility-first responsive classes for mobile (360px) through desktop (1920px) screens.
- **Client Routing:** React Router v6 implementing declarative route hierarchies, protected route wrappers (`<ProtectedRoute />`), and redirects.
- **State Management:**
  - `AuthContext`: Manages current user profile state, active JWT token persistence in `localStorage`, and logout cleanup.
  - Component State: Manages UI toggles, modal dialogs, search queries, and local loading skeletons.
- **HTTP Abstraction:** Dedicated Axios client instance (`frontend/src/services/api.ts`) with request interceptors automatically attaching `Authorization: Bearer <token>` and response interceptors handling 401 unauthorization redirects.

3.2 Tier 2: Application / API Layer (Node.js & Express)

- **Runtime:** Node.js (v18+ LTS) utilizing ECMAScript Modules (`import` / `export`).
- **Framework:** Express.js (`4.19.2`) providing unopinionated routing under the `/api/v1` namespace.
- **Controllers (`backend/src/controllers/`):**
  - Extract route parameters, query parameters, and JSON payloads.
  - Execute schema validation using Zod schemas (`backend/src/validations/`).
  - Delegate execution to domain services.
  - Wrap domain results in the standard JSON API envelope before sending HTTP response.
- **Domain Services (`backend/src/services/`):**

- Implement business calculations (e.g., compatibility score weighted formula, schedule conflict intervals).
- Execute password hashing via `bcryptjs` and token signing via `jsonwebtoken`.
- Perform authorization validation checks (e.g., verifying user is group owner before permitting member approvals).
- Free from any Express HTTP concepts (`req`, `res`), making services independently unit-testable.

- **Repositories (`backend/src/repositories/`):**
  - Encapsulate all SQL queries and database interactions.
  - Use parameterized statements (`$1, $2, ...`) via the `pg` connection pool.
  - Manage database transactions (`BEGIN`, `COMMIT`, `ROLLBACK`) for operations requiring multiple related mutations.

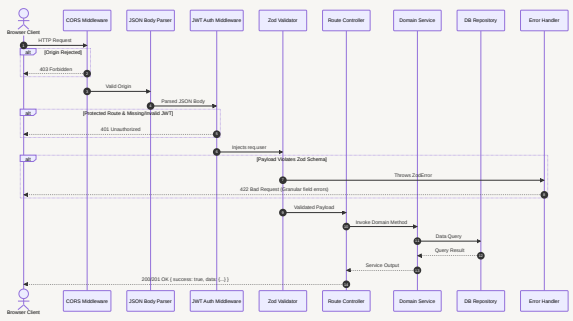
3.3 Tier 3: Persistence Layer (PostgreSQL 14+)

- **Database Engine:** PostgreSQL 14+ Relational Database Management System.
- **Normalization:** Fully normalized to Third Normal Form (3NF) to eliminate data redundancy.
- **JSONB Temporal Storage:** Weekly student availability schedules are stored as structured `JSONB` arrays inside `user_profiles.availability`, indexed via Generalized Inverted Indexes (GIN) for high-speed interval searches.
- **Relational Integrity:**
  - Foreign key constraints with `ON DELETE CASCADE` ensure child records (memberships, sessions, resources) are cleaned up automatically upon group deletion.
  - Composite unique constraint `uq_group_user` on `group_memberships(group_id, user_id)` guarantees a student cannot submit duplicate applications to the same study group.

---

4. Internal Middleware Pipeline & Request Lifecycle

Every incoming HTTP request traverses a deterministic sequential pipeline before reaching application business logic:



5. Data Flow & Interface Protocols

5.1 Client-Server Communication Contract

All API communication is executed over HTTPS using JSON request and response bodies.

Standard Response Envelope Specification:

```
{
  "success": true,
  "data": {
    "group": {
      "id": "c1f7b820-2f64-4e2a-8d2a-1a2b3c4d5e6f",
      "name": "CS301 Distributed Systems Group",
      "maxMembers": 6,
```

```
      "memberCount": 4
    }
  },
  "error": null,
  "meta": {
    "timestamp": "2026-10-16T12:00:00.000Z"
  }
}
```

Standard Error Envelope Specification:

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "GROUP_CAPACITY_REACHED",
    "message": "Cannot approve applicant: study group has reached its maximum capacity
limit of 6 members.",
    "details": []
  },
  "meta": {
    "timestamp": "2026-10-16T12:00:00.000Z"
  }
}
```

---

6. Cross-Cutting Architectural Concerns

6.1 Stateless Authentication & Authorization

- **Token Mechanism:** Stateless JSON Web Tokens signed with HMAC-SHA256 (`HS256`) containing `{ userId, email, role }`.
- **Token Lifecycle:** 24-hour expiration window; expired tokens require re-authentication.
- **Authorization Guard:** Protected endpoints enforce ownership checks (e.g., verifying `group.creator_id === req.user.userId` before modifying group settings).

6.2 Data Integrity & Concurrency Safety

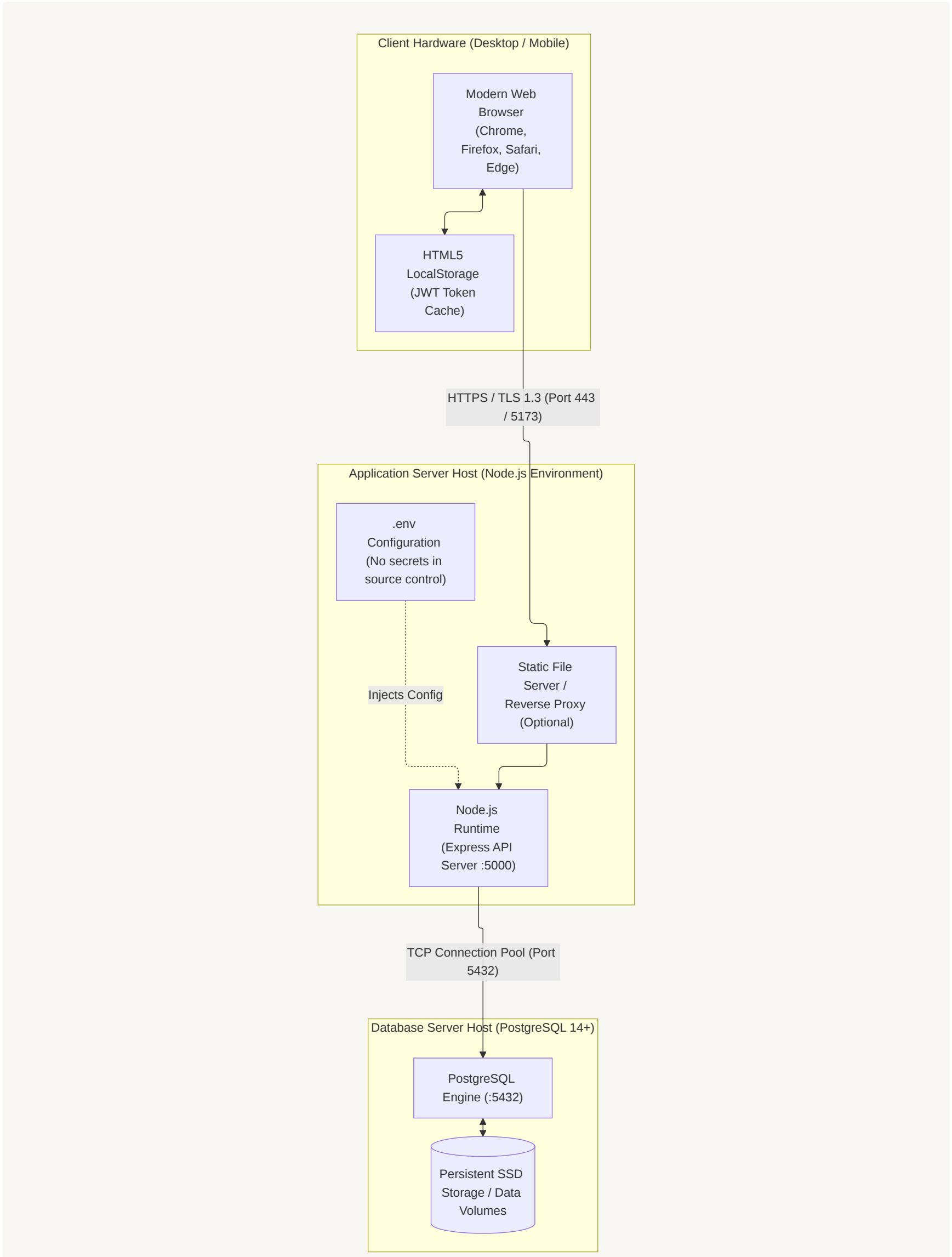
- **Row-Level Transaction Locking:** When an owner approves a new member, the repository initiates an atomic transaction using `SELECT ... FOR UPDATE` on the group row to count current active members. This eliminates race conditions during concurrent approval clicks.
- **Parameterized SQL:** All SQL queries use positional parameters (`$1, $2, ...`) ensuring complete immunity from SQL injection attacks.

6.3 Error Handling & Process Isolation

- Top-level Express error middleware catches all synchronous and asynchronous exceptions.
- Unhandled operational errors are logged with full stack traces internally while returning a sanitized `500 Internal Server Error` envelope to clients, ensuring zero internal stack or credential leaks.

---

7. Physical Deployment & Runtime Topology


Client Hardware (Desktop / Mobile)
Modern Web Browser (Chrome, Firefox, Safari, Edge)
HTML5 LocalStorage (JWT Token Cache)
HTTPS / TLS 1.3 (Port 443 / 5173)
Application Server Host (Node.js Environment)
.env Configuration (No secrets in source control)
Injects Config
Static File Server / Reverse Proxy (Optional)
Node.js Runtime (Express API Server :5000)
TCP Connection Pool (Port 5432)
Database Server Host (PostgreSQL 14+)
PostgreSQL Engine (:5432)
Persistent SSD Storage / Data Volumes